Lecture 14

Arrays+Pointers, Strings

Slides – Vybihal & Bérubé-Vallières © 2023

Arrays+Pointers

Slides – Vybihal & Bérubé-Vallières © 2023

Arrays and memory

- An array is an aggregation of items of the same type
- Its elements are placed sequentially in memory
- The compiler reserves sizeof(type)*length bytes in memory starting from the base address

```
int a[5] = {4, 3, 5, 1, 2}; // Let's say that the compiler
                            // stores this with base address 104
                            // and sizeof(int) is 4, then
                            // 4*5=20 bytes are reserved
```

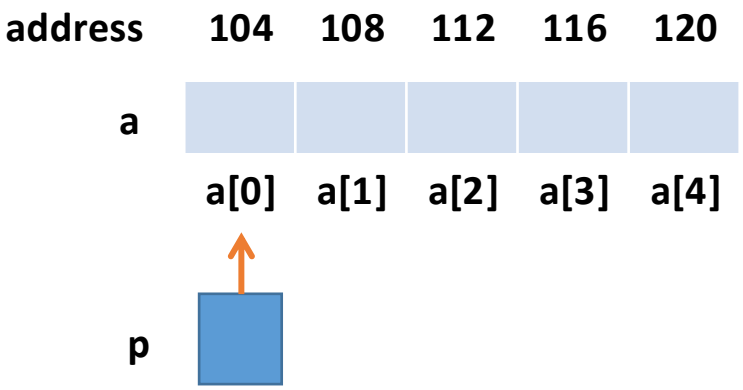
address	104	108	112	116	120 (to 123)
a	4	3	5	1	2
	a[0]	a[1]	a[2]	a[3]	a[4]

Slides – Vybihal & Bérubé-Vallières © 2023

Pointer to array

You can point to an array by pointing to the address of its first element:

```
int a[5]; // let's say that this is stored at address 104
int *p;
p = &a[0];
```



You can also use this syntax:

```
p = a;
```

What??

Array-to-Pointer Decay

- Array declarations ask the compiler to reserve some space.
  - Space on the stack for local arrays.
  - Space in global segment for global arrays.
- Anywhere else that you use the name of the array, it represents the *pointer value* that points to the array's first element.
  - This is "decay" because we lose info. int[5] becomes just int*.
- There are 3 exceptions (no need to know):
  - Does not decay as operand of &, operand of sizeof, or when the array is a string literal used as an array initializer.

Slides – Vybihal & Bérubé-Vallières © 2023

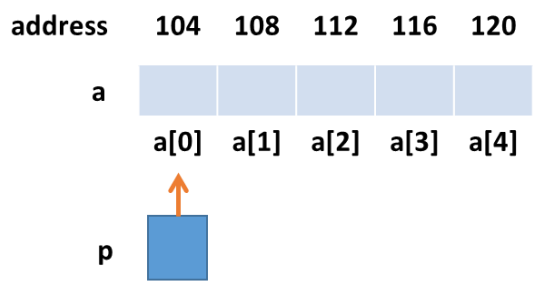
Pointer arithmetic

- You can add an integer to a pointer
  - You get a pointer
- You can subtract an integer from a pointer
  - You get a pointer
- You can subtract a pointer from a pointer (if both point to the same array)
  - You get an integer
  - Result is such that p + (q – p) is q.
  - "Point to same array" is a requirement for the result to be defined. Otherwise, it is undefined behavior. Usually, it works as you might expect, but relying on it is dangerous.

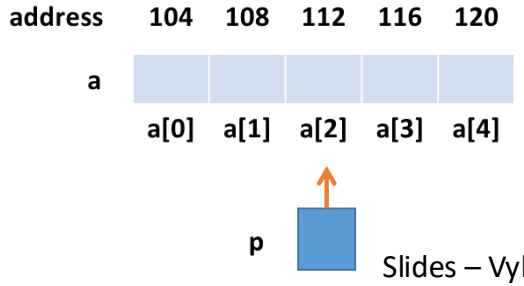
Adding an integer to a pointer

When you add an integer to a pointer, the address is increased by the integer times the size of the type the pointer points to. This means that for an array, each increase by one makes the pointer point to the next element.

```
int a[5]; // let's say that this is stored at address 104
int *p;
p = a;
```



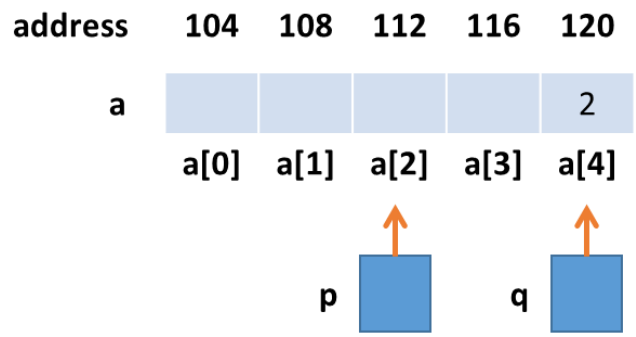

```
p += 2;
```



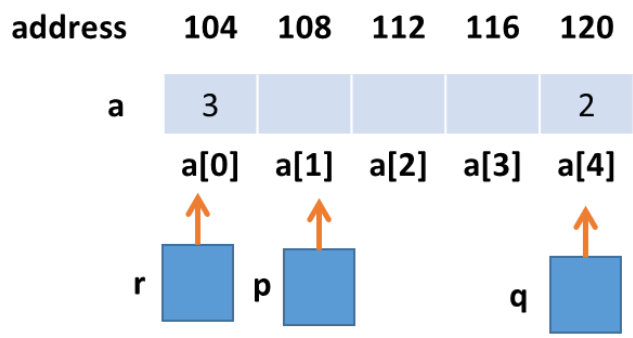
In other words, if p points to a[x], then p+i points to a[x+i]

More examples

```
int a[5]; // let's say that this is stored at address 104
int *p;
int *q;
p = &a[2];
q = p + 2;
*q = 2;
```




```
// works the same way for subtraction
// if p points to a[x], then p - i points to a[x-i]
int *r;
p -=1
r = p - 1;
*r = 3;
```




```
// getting the distance between two pointers from same array
ptrdiff_t diff = q - p // diff is now 3. p-q would be -3
```

ptrdiff_t is an alias for the integer type of the right size for this machine to store a difference between pointers. Its format specifier is %td.

Relationship between arrays and pointers

Let a be an array and i be an integer

- *(a+i) is equivalent to a[i] in **every** way
- a+i is equivalent to &a[i]
  - a+i == &a[0]+i == &a[0+i] == &a[i]

```
// Using an array name as a pointer
int a[5];
*a = 4; // equivalent to a[0] = 4
*(a+3) = 6; // equivalent to a[3] = 6

// That means we can use array index syntax with any pointers
int *p = a
p[2] = 4 // equivalent to *(p+2) = 4, in this case same as a[2] = 4
```

Difference between arrays and pointers (1/2)

When declaring

```
int a[5];
```

address	104	108	112	116	120
a					
	a[0]	a[1]	a[2]	a[3]	a[4]

The compiler allocates space for 5 integers

When declaring

```
int *p;
```

p

The compiler allocates space for a pointer but **no data**

As such,

```
*a = 5; // works
```

address	104	108	112	116	120
a	5				
	a[0]	a[1]	a[2]	a[3]	a[4]

p → **??? = 5 ... undefined behavior**

```
*p = 5; // wrong & dangerous
```

Difference between arrays and pointers (2/2)

Unlike pointers, you cannot assign a new value to an array name

```
int a[5];
int *p;
p = a;
p++; // p now points to a[1];
a++; // Wrong! Compile-time error
```

- a decays to the pointer value &a[0], but there is no pointer *variable* a to increment.

Strings

String literals

- Sequence of characters between double quotes.
- When a string literal of size n is written, the compiler stores it in some place in memory (supposed to be read-only) as a character array of size n+1 (the n+1 character is the null character, which is `'\0'`) .
  - "abcd" is stored as

address	**200**	**201**	**202**	**203**	**204**
"**abcd**"	a	b	c	d	\0

  - "" is stored as

address	**300**
""	\0

(these are just example addresses)

- Because this is an array, it can be subscripted
  - char c = "abcd"[1] // c = 'b'

String literal and assignments

- A string literal can be assigned to a character pointer, in which case the pointer points to its location in memory (first character of course)

```
char *p = "abcd"
```

address	200	201	202	203	204
"abcd"	a	b	c	d	\0

(assumed to be read-only)

p

A string literal can also be assigned to a character array, in which case it is **copied** to the array

```
char a[5] = "abcd"
```

address	200	201	202	203	204
"abcd"	a	b	c	d	\0

(read-only)

address	100	101	102	103	104
a	a	b	c	d	\0

(can be edited)

More on assigning character arrays

```
char a[5] = "abcd"
// is equivalent to:
char a[5] = {'a','b','c','d','\0'}
```

address	100	101	102	103	104
a	a	b	c	d	\0

Similarly to using array initializers, assigning string literals to arrays follows these rules:

- If the string literal is shorter than the specified size, the remaining characters are filled with null terminators

```
char a[5] = "ab"
// is equivalent to:
char a[5] = {'a','b','\0','\0','\0'}
// which is a string of size 2 since a string stops at the null terminator
```

address	100	101	102	103	104
a	a	b	\0	\0	\0

- Since the compiler can deduce the size of the array from the length of the string literal, it can be omitted.
  - This doesn't mean you can assign to a longer string later! The compiler reserves space for that length exactly when declared.

```
char a[] = "abcd"
```

address	100	101	102	103	104
a	a	b	c	d	\0

(Not) Editing String Literals (1/2)

```
char *p = "abcd"
```

makes a pointer to an array at a location in memory that might be read-only. As such, editing the string is undefined behaviour and should not be done.

```
char p[0] = 'd' // Wrong!
```

To illustrate this, it is good practice to make it a constant:

```
const char *p = "abcd‟ // const char, not const pointer
```

Note that this doesn't mean the pointer itself can't be made to point elsewhere

```
p = "hello world" // Allowed!
```

String literals can't be edited but copies of them can

```
char a[] = "abcd"
a[0] = 'd' // The string a is now "dbcd"
```


Slides – Vybihal & Bérubé-Vallières © 2023

(Not) Editing String Literals (2/2)

For the same reason, you could not do something like this

```
char *p;

fgets(p, 5, stdin); // Wrong! p does not point to anything
                    // yet, so this undefined behaviour

// but even if it did point to a string literal…

p = "abcd";

fgets(p, 5, stdin); // Wrong! Now p indeed points to a string
                       literal, but it is in read-only
                       memory!

// instead..

char a[5];

fgets(a, 5, stdin); // Correct!

p = a;

fgets(p, 5, stdin); // Also works!
```


Slides – Vybihal & Bérubé-Vallières © 2023

Null terminator

Strings in C are null-terminated (end with the character `'\0'`)

Functions have no other way of knowing when the string ends!

When using a function like printf, it prints every single character until it finds a null character. If you were to remove the null character:

```
char str[] = "abcd"
str[4] = 'e' // Wrong!
printf("%s", str); // will keep printing until it finds null
   terminator in memory or segfaults!
```

Reading strings

- Note that when using scanf (or other functions) with arrays or strings, you do not have to add & in front since it is already treated like a pointer in that context

```
char str[100];
scanf("%s", str); // but scanf("%s", &str); also works
```

- All standard library functions to read strings usually add a null terminator at the end **but check man page**
  - Those that read from standard input will generally also read the newline from the user, but as mentioned previously scanf does not

```
char str1[100];
char str2[100];
// Assume that for both of these
// user writes Bob then presses enter
fgets(str1, 100, stdin);
scanf("%s", str2);
```

address	100	101	102	103	104	…
str1	B	o	b	\n	\0	

address	200	201	202	203	…
str2	B	o	b	\0	

Accessing characters

Since strings are arrays, just like we saw earlier we can access its characters using array or pointer operations

```
char str[] = "abcd";
str[2] = 'z';
*(str+1) = 'y';
printf("%s\n", str); // outputs ayzd
```

How **Not** to Compare Strings (and arrays)

```
char str1[] = "abcd";
char str2[] = "abcd";

if (str1 == str2)
   puts("True");
else
   puts("False");
// Might output false because it compares addresses of str1 and str2
// Be careful:
if (*str1 == *str2)
   puts("True");
else
   puts("False");
// Outputs true, but only  because the first characters are the
   same! *str1 dereferences the address of the first element
```


Slides – Vybihal & Bérubé-Vallières © 2023

String.h

Important Functions

- int strcmp(char *s1, char *s2)
  - Compare two strings
- int strncmp(char *s1, char *s2, size_t num)
  - Compare the first num characters of two strings
- size_t strlen(char *string)
  - Returns the length of the string (does not include null terminator)
- char *strcpy(char *dest, char *src) … strncpy
  - Copies the src string into the dest array. Does not pad (if dest already has values and src is shorter, the values stay there after the null terminator). Make sure that dest is long enough, otherwise buffer overflow!
    - strncpy copies the first N characters. If src is shorter than N it does pad with null characters. If src is longer, **no null terminator.** The problem remains that dest should be long enough.
- char *strcat(char *dest, char *src) … strncat
  - Append the src string to dest str. Dest must be long enough efor new characters.
- void *memset(void *ptr, int value, size_t num)
  - Set the first num bytes at the location pointed by ptr to value

size_t is an alias for an unsigned integer type of some length depending on compiler/machine: In this case you can use int and it will be implicitly casted

The strcmp function

Calculates the difference between two strings:

- **Returns 0 when equal**
- Returns >0 when first non-matching character in first string has a greater integer value than that of the second string
- Returns <0 when first non-matching character in first string has a lower integer value than that of the second string

```
char *a="bob";
char b[] = {'b', 'o', 'b', '\0'};

if (strcmp(a,b)==0) // then it is the same
if (strcmp(a,b)!=0) // then it is not the same

if (strcmp(a,b)) // avoid this: will be true if NOT the same
// (which means that if (!strcmp(a,b)) is what you want for
// it to be true for same, but hard to read…
```

Examples

```
int x = strcmp("bob", "bob");      //     0
int x = strcmp("bob", "mary");     //    <0
int x = strcmp("mary", "bob");     //    >0

int x = strncmp("mark", "mary",3); //  0

int x = strlen("mary");            // x = 4

char array[100];
strcpy(array, "first words");
strcat(array, "second words");
printf("%s", array);               // outputs: first wordssecond words

char array2[100];
memset(array2, '*', 50);           // first 50 cells are *
```

Implementing string.h functions using pointers.

```
int strlen(char *str)
{
        int i;

        for(i=0; *str != '\0'; i++, str++);

        return i;
}
```

```
int strcmp(char *s1, char *s2)
{
        for(; *s1!='\0' && *s2!='\0' && *s1==*s2; s1++,s2++);

        return *s1-*s2;
}
```

#include <string.h>

```
void    *memccpy(void *restrict, const void *restrict, int, size_t);

void    *memchr(const void *, int, size_t);
int     memcmp(const void *, const void *, size_t);
void    *memcpy(void *restrict, const void *restrict, size_t);
void    *memmove(void *, const void *, size_t);
void    *memset(void *, int, size_t);
char    *strcat(char *restrict, const char *restrict);
char    *strchr(const char *, int);   <----------
int     strcmp(const char *, const char *);
int     strcoll(const char *, const char *);
char    *strcpy(char *restrict, const char *restrict);
size_t  strcspn(const char *, const char *);
char    *strdup(const char *);   <----------
char    *strerror(int);
int     *strerror_r(int, char *, size_t);
size_t  strlen(const char *);
char    *strncat(char *restrict, const char *restrict, size_t);
int     strncmp(const char *, const char *, size_t);
char    *strncpy(char *restrict, const char *restrict, size_t);
char    *strpbrk(const char *, const char *);
char    *strrchr(const char *, int);
size_t  strspn(const char *, const char *);
char    *strstr(const char *, const char *);   <----------
char    *strtok(char *restrict, const char *restrict);
char    *strtok_r(char *, const char *, char **);
size_t  strxfrm(char *restrict, const char *restrict, size_t);
```

#include<ctype.h>

- Case manipulation:
  - int c = toupper(int);
  - int c = tolower(int);
- Character testing:
  - int x = isalpha(int);
  - int x = isalphanum(int);
  - int x = isdigit(int);

```
if(toupper(c)=='X')
```

```
if(isalpha(c))
```

Note: char converts implicitly to int.

```
<ctype.h>:
        int isalnum(int c);
        int isalpha(int c);
        int iscntrl(int c);
        int isdigit(int c);
        int isgraph(int c);
        int islower(int c);
        int isprint(int c);
        int ispunct(int c);
        int isspace(int c);
        int isupper(int c);
        int isxdigit(int c);
        int tolower(int c);
        int toupper(int c);
```

Counting Characters

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

int main(void)
{
    char message[] = "hello, 123";
    int digits=0, letters=0, other=0, i;
    char c;

    for(i=0; i<strlen(message); i++)
    {
        c = message[i];
        if (isalpha(c)) letters++;
        else if (isdigit(c)) digits++;
        else other++;
    }
    // letters = 5
    // digits = 3
    // other = 2
}
```